1 Разбор заданий к лекции 02: Ограниченные деревья поиска и анализ ветвления

Условия. Авторские решения; номера совпадают с заданиями.

1.1 02.1

Для $(1,2)$: $\rho^{-1}+\rho^{-2}=1$, откуда $\rho=(1+\sqrt{5})/2\approx 1.618034$. Для $(2,2,2)$: $3\rho^{-2}=1$, откуда $\rho=\sqrt{3}\approx 1.732051$. При общей мере первая оценка лучше. Выбрав C для граничных значений, из $L(j)\le C\rho^j$ для меньших j выводим $L(k)\le C\rho^k\sum_i \rho^{-a_i}=C\rho^k$.

Если первая мера исходно равна $2k$, а вторая k, сравнивать нужно 1.618034^{2k} и 1.732051^k, и вывод меняется.

1.2 02.2

В объединении клик индуцированного P_3 нет. Обратно, если компонента не клика, выберем две несмежные вершины и кратчайший путь между ними. Первые три вершины образуют индуцированный P_3: хорда сократила бы путь. Значит, отсутствие P_3 эквивалентно требуемой структуре.

Найдём тройку a,b,c с рёбрами ab,bc и без ac за $O(n^3)$. Любое допустимое множество удаления пересекает эту тройку, поскольку при сохранении всех трёх индуцированный P_3 остаётся. Ветвимся на удаление каждой вершины, уменьшая бюджет. При отрицательном бюджете NO, при отсутствии препятствия YES. Индукция по бюджету доказывает корректность; дерево имеет $O(3^k)$ узлов. Время $O(3^k n^3)$, память полиномиальна при поиске в глубину.

1.3 02.3

На одном ребре uv при $k=2$ покрытие $\{u,v\}$ попадает в обе ветви. Для непересекающегося разбиения берём произвольную неизолированную v: в первой ветви $v\in S$, во второй $v\notin S$, и тогда $N(v)\subseteq S$. Первая ветвь — $(G-v,k-1)$, вторая — $(G-N[v],k-\deg(v))$. Добавление фиксированных выбранных вершин является биекцией между решениями дочерних задач и соответствующей частью решений родителя. Части не пересекаются по условию на v.

Если рёбер нет, ответ не 1, а $\sum_{i=0}^{\min(k,n)}\binom{n}{i}$ при $k\ge 0$. При $k<0$ ответ 0. Изолированные вершины нельзя просто удалять без учёта их выбора. Числа имеют $O(n)$ бит, поэтому арифметика добавляет лишь полиномиальный множитель.

1.4 02.4

Состояние — множество S переменных, уже получивших 1; остальные пока 0. При $|S|>k$ отвергаем. Если формула удовлетворена, возвращаем S. Иначе выбираем неудовлетворённый дизъюнкт C. Любой отрицательный литерал $\neg x$ в нём имеет $x\in S$ и навсегда останется ложным при дальнейшем добавлении единиц. Исправить C можно только положительным литералом x с $x\notin S$. По всем таким различным переменным ветвимся, добавляя одну в S. Если их нет, продолжение невозможно; это также обрабатывает пустой дизъюнкт.

Для полноты зафиксируем удовлетворяющее множество T размера до k. Изначально $S=\varnothing\subseteq T$. Пока $S\subseteq T$, неудовлетворённый при S дизъюнкт должен иметь положительную переменную из $T\setminus S$: отрицательные литералы из уже выбранных переменных не помогут и при T. Значит, одна ветвь сохраняет $S\subseteq T$. Алгоритм либо раньше найдёт ответ, либо

дойдёт до T. Выдаваемые множества проверяются непосредственно. Ветвей не больше d, глубина до $k+1$, работа на узел полиномиальна, что даёт заявленное время. Установка новой единицы может испортить другие дизъюнкты; это не мешает, поскольку в следующем вызове проверяется вся формула.

1.5 02.5

Храним (H, ℓ, b), где ℓ — остаток количества, b — остаток стоимости. При любом отрицательном бюджете FAIL; при пустом множестве рёбер возвращаем пустое множество. Для ребра uv вызываем $(H-u, \ell-1, b-w(u))$ и $(H-v, \ell-1, b-w(v))$, при успехе добавляем выбранный конец.

Любое допустимое покрытие содержит один из концов; его удаление уменьшает число и стоимость на соответствующие величины. Обратно, восстановление соблюдает оба ограничения. Неотрицательность весов позволяет безопасно отвергать отрицательный остаток стоимости: дальнейший выбор его не исправит. Количественный бюджет уменьшается в каждой ветви, отсюда $O(2^k)$ узлов независимо от весов. Вычитания и сравнения используют $O(L + \log(n+1))$ бит для весов длины не больше L и суммы; это полиномиально по записи входа. Если контролировать глубину только стоимостью, выбор нулевой вершины не уменьшает меру; тогда глубина не ограничена B.